Shri Tontada Siddhalingeshwar Kalyana Kendra's

Tontadarya College of Engineering

Mundargi Road, Gadag – 582 101, Karnataka State
Approved by AICTE, New Delhi
Affiliated to Visvesvaraya Technological University, Belagavi Accredited by NBA (CSE, CV, ECE & EEE) and NAAC (B)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

LABORATORY MANUAL

COMPUTER NETWORKS LAB

(BCS502)

Semester: V

Year: 2024 – 25 (Odd Sem)

Faculty in-charge

Mr. A K KULKARNI

Assistant Professor

Department Of Computer Science & Engineering

Tontadarya College of Engineering

Gadag

Laboratory Experiments

1. Implement three nodes point – to – point network with duplex links between them. Set the queue size, vary the bandwidth, and find the number of packets dropped.

2. Implement transmission of ping messages/trace route over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion.

3. Implement an Ethernet LAN using n nodes and set multiple traffic nodes and plot congestion window for different source / destination.

4. Develop a program for error detecting code using CRC-CCITT (16- bits).

5. Develop a program to implement a sliding window protocol in the data link layer.

6. Develop a program to find the shortest path between vertices using the BellmanFord and path vector routing algorithm.

7. Using TCP/IP sockets, write a client – server program to make the client send the file name and to make the server send back the contents of the requested file if present.

8. Develop a program on a datagram socket for client/server to display the messages on client side, typed at the server side.

9. Develop a program for a simple RSA algorithm to encrypt and decrypt the data.

10. Develop a program for congestion control using a leaky bucket algorithm

Network Simulator (NS)

Overview

NS (i.e., version 2) is an object-oriented discrete event developed at UC Berkeley written in C++ and OTcl (Tcl script language with Object-oriented extensions developed at MIT) is available. NS is primarily useful for simulating local and wide area networks. Although NS is fairly easy to use once you get to know the simulator.

NS simulates variety of IP networks. It implements network protocols such as TCP and UPD, traffic source behavior such as FTP, Telnet, Web, CBR and VBR, router queue management mechanism such as Drop Tail, RED and CBQ, routing algorithms such as Dijkstra, and more. NS also implements multicasting and some of the MAC layer protocols for LAN simulations. The NS project is now a part of the VINT project that develops tools for simulation results display, analysis and converters that convert network topologies generated by well-known generators to NS formats.

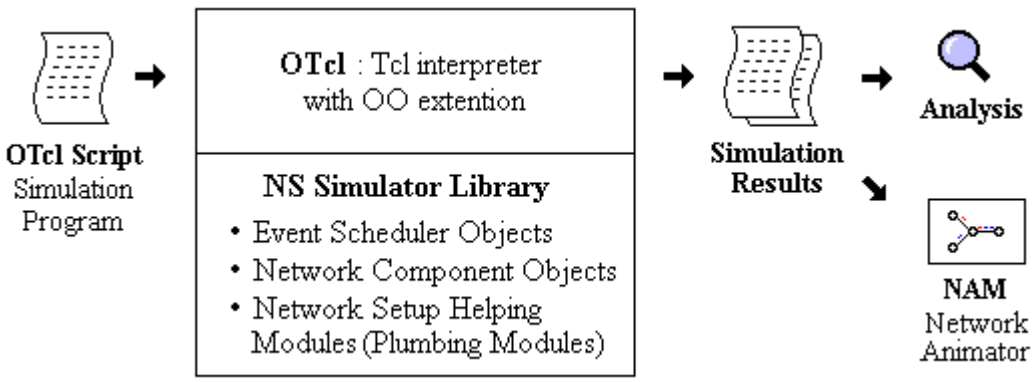


Figure 1: Simplified User's View of NS.

As shown in **Figure 1**, NS is Object-oriented Tcl (OTcl) script interpreter that has a simulation event scheduler and network component object libraries, and network setup (plumbing) module libraries (actually,

plumbing modules are implemented as member functions of the base simulator object). In other words, to use NS, you program in OTcl script language. To setup and run a simulation network, a user should write an OTcl script that initiates an event scheduler, sets up the network topology using the network objects and the plumbing functions in the library, and tells traffic sources when to start and stop transmitting packets through the event scheduler.

The term "plumbing" is used for a network setup, because setting up a network is plumbing possible data paths among network objects by setting the "neighbor" pointer of an object to the address of an appropriate object. When a user wants to make a new network object, he or she can easily make an object either by writing a new object or by making a compound object from the object library, and plumb the data path through the object. This may sound like complicated job, but the plumbing OTcl modules actually make the job very easy. The power of NS comes from this plumbing.

Another major component of NS beside network objects is the event scheduler. An event in NS is a packet ID that is unique for a packet with scheduled time and the pointer to an object that handles the event. In NS, an event scheduler keeps track of simulation time and fires all the events in the event queue scheduled for the current time by invoking appropriate network components, which usually are the ones who issued the events, and let them do the appropriate action associated with packet pointed by the event. Network components communicate with one another passing packets, however this does not consume actual simulation time. All the network components that need to spend some simulation time handling a packet (i.e. need a delay) use the event scheduler by issuing an event for the packet and waiting for the event to be fired to itself before doing further action handling the packet. For example, a network switch component that simulates a switch with 20 microseconds of switching delay issues an event for a packet to be switched to the scheduler as an event 20 microsecond later. The scheduler after 20 microsecond dequeues the event and fires it to the switch component, which then passes the packet to an appropriate output link component. Another use of an event scheduler is timer. For example, TCP needs a timer to keep track of a packet transmission time out for retransmission (transmission of a packet with the same TCP packet number but different NS packet ID). Timers use event schedulers in a similar manner that delay does. The only difference is that timer measures a time value associated with a packet and does an appropriate action related to that packet after a certain time goes by, and does not simulate a delay.

NS is written not only in OTcl but in C++ also. For efficiency reason, NS separates the data path implementation from control path implementations. In order to reduce packet and event processing time (not

simulation time), the event scheduler and the basic network component objects in the data path are written and compiled using C++. These compiled objects are made available to the OTcl interpreter through an OTcl linkage that creates a matching OTcl object for each of the C++ objects and makes the control functions and the configurable variables specified by the C++ object act as member functions and member variables of the corresponding OTcl object. In this way, the controls of the C++ objects are given to OTcl. It is also possible to add member functions and variables to a C++ linked OTcl object. The objects in C++ that do not need to be controlled in a simulation or internally used by another object do not need to be linked to OTcl. Likewise, an object (not in the data path) can be entirely implemented in OTcl. **Figure 2** shows an object hierarchy example in C++ and OTcl. One thing to note in the figure is that for C++ objects that have an OTcl linkage forming a hierarchy, there is a matching OTcl object hierarchy very similar to that of C++.

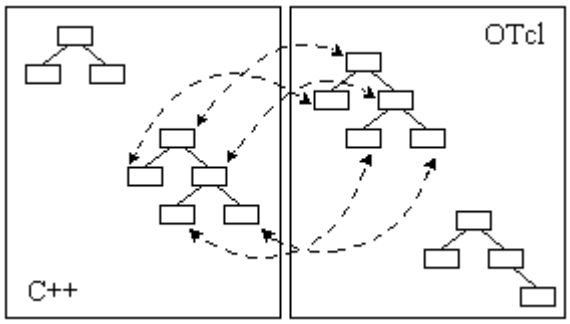


Figure 2: C++ and OTcl: The Duality

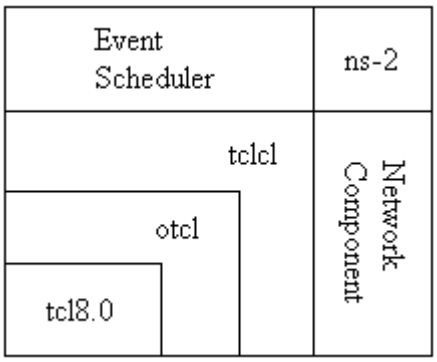


Figure 3: Architectural View of NS

Figure 3 shows the general architectural view of NS. In this figure a general user (not an NS developer) can be thought of standing at the left bottom corner, designing and running simulations in Tcl using the simulator objects in the OTcl library. The event schedulers and most of the network components are implemented in C++ and available to OTcl through an OTcl linkage that is implemented using tclcl. The whole thing together makes NS, which is an OO extended Tcl interpreter with network simulator libraries.

This section briefly examined the general structure and architecture of NS. At this point, one might be wondering about how to obtain NS simulation results. As shown in **Figure 1**, when a simulation is finished, NS produces one or more text-based output files that contain detailed simulation data, if specified to do so in the input Tcl (or more specifically, OTcl) script. The data can be used for simulation analysis (two simulation result analysis examples are presented in later sections) or as an input to a graphical simulation display tool called Network Animator (NAM) that is developed as a part of VINT project. NAM has a nice graphical user interface similar to that of a CD player (play, fast forward, rewind, pause and so on), and also has a display speed controller. Furthermore, it can graphically present information such as throughput and number of packet drops at each link, although the graphical information cannot be used for accurate simulation analysis.

OTcl: The User Language

As mentioned in the overview section, NS is basically an OTcl interpreter with network simulation object libraries. It is very useful to know how to program in OTcl to use NS. This section shows an example Tcl and OTcl script, from which one can get the basic idea of programming in OTcl. These examples are from the 5th VINT/NS Simulation Tutorial/Workshop. This section and the sections after assumes that the reader installed NS, and is familiar with C and C++.

Example 1 is a general Tcl script that shows how to create a procedure and call it, how to assign values to variables, and how to make a loop. Knowing that OTcl is Object-orieneted extension of Tcl, it is obvious that all Tcl commands work on OTcl - the relationship between Tcl and Otcl is just same as C and C++. To run this script you should download **ex-tcl.tcl**, and type "ns ex-tcl.tcl" at your shell prompt the command "ns" starts the NS (an OTcl interpreter). You will also get the same results if you type "tcl extcl.tcl", if tcl8.0 is installed in your machine.

```
# Writing a procedure called "test"
proc test {} {
    set a 43
    set b 27
    set c [expr $a + $b]
    set d [expr [expr $a - $b] * $c]
    for {set k 0} {$k < 10} {incr k} {
        if {$k < 5} {
            puts "k < 5, pow = [expr pow($d, $k)]"
        } else {
            puts "k >= 5, mod = [expr $d % $k]"
        }
    }
}

# Calling the "test" procedure created above
test
```

Example 1: A Sample Tcl Script

In Tcl, the keyword proc is used to define a procedure, followed by an procedure name and arguments in curly brackets. The keyword set is used to assign a value to a variable. [expr ...] is to make the interpreter calculate the value of expression within the bracket after the keyword. One thing to note is that to get the value assigned to a variable, $ is used with the variable name. The keyword puts prints out the following string within double quotation marks. The following shows the result of **Example 1**.

```
k < 5, pow = 1.0
k < 5, pow = 1120.0
k < 5, pow = 1254400.0
k < 5, pow = 1404928000.0
k < 5, pow = 1573519360000.0
k >= 5, mod = 0
k >= 5, mod = 4
k >= 5, mod = 0
k >= 5, mod = 0
k >= 5, mod = 4
```

The **next example** is an object-oriented programming example in OTcl. This example is very simple, but shows the way which an object is created and used in OTcl. As an ordinary NS user, the chances that you will write your own object might be rare. However, since all of the NS objects that you will use in a NS simulation programming, whether or not they are written in C++ and made available to OTcl via the linkage or written only in OTcl, are essentially OTcl objects, understanding OTcl object is helpful.

```
# add a member function call "greet"
Class mom
mom instproc greet {} {
    $self instvar age_
    puts "$age_ year old mom say:
    How are you doing?"
}

# Create a child class of "mom" called "kid"
# and overide the member function "greet"
Class kid -superclass mom
kid instproc greet {} {
    $self instvar age_
    puts "$age_ year old kid say:
    What's up, dude?"
}

# Create a mom and a kid object, set each age
set a [new mom]
$a set age_ 45
set b [new kid]
$b set age_ 15

# Calling member function "greet" of each object
$a greet
$b greet
```

Example 2: A Sample OTcl Script

Example 2 is an OTcl script that defines two object classes, "mom" and "kid", where "kid" is the child class of "mom", and a member function called "greet" for each class. After the class definitions, each object instance is declared, the "age" variable of each instance is set to 45 (for mom) and 15 (for kid), and the "greet" member function of each object instance is called. The keyword Class is to create an object class and instproc is to define a member function to an object class. Class inheritance is specified using the keyword -superclass. In defining member functions, $self acts same as the "this" pointer in C++, and instvar checks if the following variable name is already declared in its class or in its superclass. If the variable name given is already declared, the variable is referenced, if not a new one is declared. Finally, to create an object instance, the keyword new is used as shown in the example. Downloading ex-otcl.tcl and executing "ns ex-otcl.tcl" will give you the following result:

```
45 year old mom say:
    How are you doing?
15 year old kid say:
    What's up, dude?
```

Simple Simulation Example

This section shows a simple NS simulation script and explains what each line does. **Example 3** is an OTcl script that creates the simple network configuration and runs the simulation scenario in **Figure 4**.

To run this simulation type "ns program_name.tcl" at shell prompt.

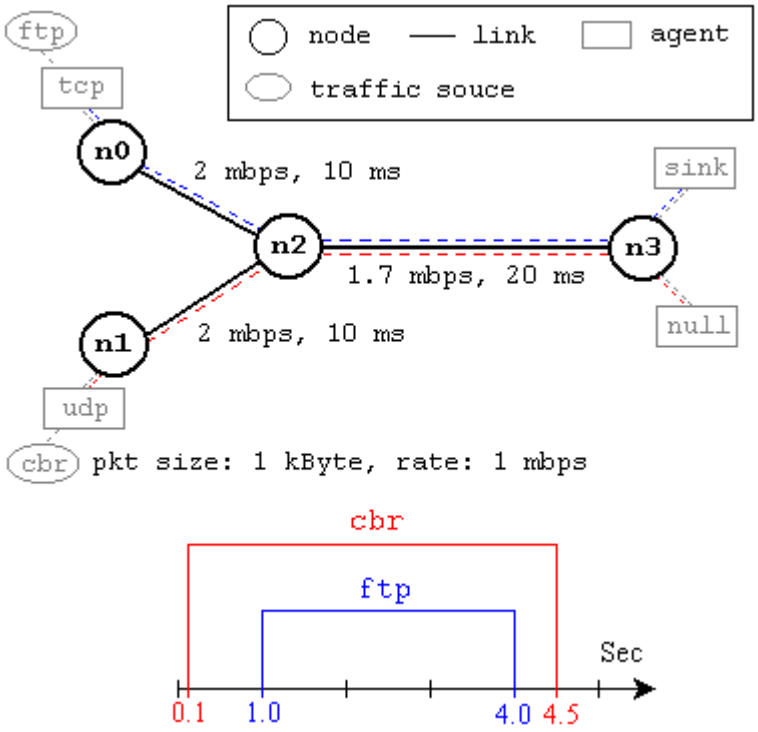


Figure 4: A Simple Network Topology and Simulation Scenario

This network consists of 4 nodes (n0, n1, n2, n3) as shown in above figure. The duplex links between n0 and n2, and n1 and n2 have 2 Mbps of bandwidth and 10 ms of delay. The duplex link between n2 and n3 has 1.7 Mbps of bandwidth and 20 ms of delay. Each node uses a DropTail queue, of which the maximum size is 10. A "tcp" agent is attached to n0, and a connection is established to a tcp "sink" agent attached to n3. As default, the maximum size of a packet that a "tcp" agent can generate is 1KByte. A tcp "sink" agent generates and sends ACK packets to the sender (tcp agent) and frees the received packets. A "udp" agent that is attached to n1 is connected to a "null" agent attached to n3. A "null" agent just frees the packets received. A "ftp" and a "cbr" traffic generator are attached to "tcp" and "udp" agents respectively, and the "cbr" is configured to generate 1 KByte packets at the rate of 1 Mbps. The "cbr" is set to start at 0.1 sec and stop at 4.5 sec, and "ftp" is set to start at 1.0 sec and stop at 4.0 sec.

```
#Create a simulator object
set ns [new Simulator]

#Define different colors for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red

#Open the NAM trace file
set nf [open out.nam w]
$ns namtrace-all $nf

#Define a 'finish' procedure
proc finish {} {
        global ns nf
        $ns flush-trace
        #Close the NAM trace file
        close $nf
        #Execute NAM on the trace file
        exec nam out.nam &
        exit 0
}

#Create four nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
  #Create links between the nodes
  $ns duplex-link $n0 $n2 2Mb 10ms DropTail
  $ns duplex-link $n1 $n2 2Mb 10ms DropTail
  $ns duplex-link $n2 $n3 1.7Mb 20ms DropTail

  #Set Queue Size of link (n2-n3) to 10
  $ns queue-limit $n2 $n3 10

  #Give node position (for NAM)
  $ns duplex-link-op $n0 $n2 orient right-down
  $ns duplex-link-op $n1 $n2 orient right-up
  $ns duplex-link-op $n2 $n3 orient right

  #Monitor the queue for link (n2-n3). (for NAM)
  $ns duplex-link-op $n2 $n3 queuePos 0.5


  #Setup a TCP connection
  set tcp [new Agent/TCP]
  $tcp set class_ 2
  $ns attach-agent $n0 $tcp
  set sink [new Agent/TCPSink]
  $ns attach-agent $n3 $sink
  $ns connect $tcp $sink
  $tcp set fid_ 1

  #Setup a FTP over TCP connection
  set ftp [new Application/FTP]
  $ftp attach-agent $tcp
  $ftp set type_ FTP
```

```
#Setup a UDP connection
set udp [new Agent/UDP]
$ns attach-agent $n1 $udp
set null [new Agent/Null]
$ns attach-agent $n3 $null
$ns connect $udp $null
$udp set fid_ 2

#Setup a CBR over UDP connection
set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 1mb
$cbr set random_ false


#Schedule events for the CBR and FTP agents
$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftp start"
$ns at 4.0 "$ftp stop"
$ns at 4.5 "$cbr stop"

#Detach tcp and sink agents (not really necessary)
$ns at 4.5 "$ns detach-agent $n0 $tcp ; $ns detach-agent $n3 $sink"

#Call the finish procedure after 5 seconds of simulation time
$ns at 5.0 "finish"

#Print CBR packet size and interval
puts "CBR packet size = [$cbr set packet_size_]"
puts "CBR interval = [$cbr set interval_]"

#Run the simulation
$ns run
```

Example 3: A Simple NS Simulation Script

The following is the explanation of the script above. In general, an NS script starts with making a Simulator object instance.

- set ns [new Simulator]: generates an NS simulator object instance, and assigns it to variable ns (italics is used for variables and values in this section). What this line does is the following:
  - Initialize the packet format (ignore this for now)
  - Create a scheduler (default is calendar scheduler)
  - Select the default address format (ignore this for now)
- The "Simulator" object has member functions that do the following:

- Create compound objects such as nodes and links (described later)
- Connect network component objects created (ex. attach-agent)
- Set network component parameters (mostly for compound objects)
- Create connections between agents (ex. make connection between a "tcp" and "sink") Specify NAM display options

Most of member functions are for simulation setup (referred to as plumbing functions in the Overview section) and scheduling, however some of them are for the NAM display. The "Simulator" object member function implementations are located in the "**ns-2/tcl/lib/ns-lib.tcl**" file.

- $ns color fid color: is to set color of the packets for a flow specified by the flow id (fid). This member function of "Simulator" object is for the NAM display, and has no effect on the actual simulation.

- $ns namtrace-all file-descriptor: This member function tells the simulator to record simulation traces in NAM input format. It also gives the file name that the trace will be written to later by the command $ns flush-trace. Similarly, the member function trace-all is for recording the simulation trace in a general format.

- proc finish {}: is called after this simulation is over by the command $ns at 5.0 "finish". In this function, post-simulation processes are specified.

- set n0 [$ns node]: The member function node creates a node. A node in NS is compound object made of address and port classifiers (described in a later section). Users can create a node by separately creating an address and a port classifier objects and connecting them together. However, this member function of Simulator object makes the job easier. To see how a node is created, look at the files: "**ns2/tcl/libs/ns-lib.tcl**" and "**ns-2/tcl/libs/ns-node.tcl**".

- $ns duplex-link node1 node2 bandwidth delay queue-type: creates two simplex links of specified bandwidth and delay, and connects the two specified nodes. In NS, the output queue of a node is implemented as a part of a link, therefore users should specify the queue-type when creating links. In the above simulation script, DropTail queue is used. If the reader wants to use a RED queue, simply replace

the word DropTail with RED. The NS implementation of a link is shown in a later section. Like a node, a link is a compound object, and users can create its sub-objects and connect them and the nodes. Link source codes can be found in "ns-2/tcl/libs/ns-lib.tcl" and "ns-2/tcl/libs/ns-link.tcl" files. One thing to note is that you can insert error modules in a link component to simulate a lossy link (actually users can make and insert any network objects). Refer to the NS documentation to find out how to do this.

- $ns queue-limit node1 node2 number: This line sets the queue limit of the two simplex links that connect node1 and node2 to the number specified. At this point, the authors do not know how many of these kinds of member functions of Simulator objects are available and what they are. Please take a look at "ns-2/tcl/libs/ns-lib.tcl" and "ns-2/tcl/libs/ns-link.tcl", or NS documentation for more information.

- $ns duplex-link-op node1 node2 ...: The next couple of lines are used for the NAM display. To see the effects of these lines, users can comment these lines out and try the simulation.

Now that the basic network setup is done, the next thing to do is to setup traffic agents such as TCP and UDP, traffic sources such as FTP and CBR, and attach them to nodes and agents respectively.

- set tcp [new Agent/TCP]: This line shows how to create a TCP agent. But in general, users can create any agent or traffic sources in this way. Agents and traffic sources are in fact basic objects (not compound objects), mostly implemented in C++ and linked to OTcl. Therefore, there are no specific Simulator object member functions that create these object instances. To create agents or traffic sources, a user should know the class names these objects (Agent/TCP, Agnet/TCPSink, Application/FTP and so on). This information can be found in the NS documentation or partly in this documentation. But one shortcut is to look at the "**ns-2/tcl/libs/ns-default.tcl**" file. This file contains the default configurable parameter value settings for available network objects. Therefore, it works as a good indicator of what kind of network objects are available in NS and what are the configurable parameters.

- $ns attach-agent node agent: The attach-agent member function attaches an agent object created to a node object. Actually, what this function does is call the attach member function of specified node, which attaches the given agent to itself. Therefore, a user can do the same thing by, for example, $n0 attach $tcp. Similarly, each agent object has a member function attach-agent that attaches a traffic source object to itself.

- $ns connect agent1 agent2: After two agents that will communicate with each other are created, the next thing is to establish a logical network connection between them. This line establishes a network connection by setting the destination address to each others' network and port address pair.

Assuming that all the network configuration is done, the next thing to do is write a simulation scenario (i.e. simulation scheduling). The Simulator object has many scheduling member functions. However, the one that is mostly used is the following:

- $ns at time "string": This member function of a Simulator object makes the scheduler (scheduler_ is the variable that points the scheduler object created by [new Scheduler] command at the beginning of the script) to schedule the execution of the specified string at given simulation time. For example, $ns at 0.1 "$cbr start" will make the scheduler call a start member function of the CBR traffic source object, which starts the CBR to transmit data. In NS, usually a traffic source does not transmit actual data, but it notifies the underlying agent that it has some amount of data to transmit, and the agent, just knowing how much of the data to transfer, creates packets and sends them.

After all network configuration, scheduling and post-simulation procedure specifications are done, the only thing left is to run the simulation. This is done by ***$ns* run**.

Program No. 1

Simulate a three nodes point-to-point network with duplex links between them. Set the queue size & vary the bandwidth & find the number of packets dropped.

p1.tcl

```
set ns [new Simulator]

# Create 3 nodes  set
        n0 [$ns node]
        set n1 [$ns node]
        set n2 [$ns node]

# Create a trace and arrange for all the trace events of the links subsequently created to be #to
"out.tr"  set f [open out.tr w]
        $ns trace-all $f  set nf
        [open out.nam w]
        $ns namtrace-all $nf

# Connect the two nodes with a 1.5Mb link with a transmission delay of 10ms using FIFO drop-tail
queuing
        $ns duplex-link $n0 $n1 1.5Mb 10ms DropTail
        $ns duplex-link $n1 $n2 1.5Mb 10ms DropTail
        $ns duplex-link $n2 $n0 1.5Mb 10ms DropTail

#set the queue limit between links
        $ns queue-limit $n0 $n1 10
        $ns queue-limit $n1 $n2 10
        $ns queue-limit $n2 $n0 10

# Set up BSD Tahoe TCP connections in opposite directions.
        set tcp_src1 [new Agent/TCP]  set
        tcp_snk1 [new Agent/TCPSink]
        set tcp_src2 [new Agent/TCP]  set
        tcp_snk2 [new Agent/TCPSink]
        set tcp_src3 [new Agent/TCP]
        set tcp_snk3 [new Agent/TCPSink]

#Attach agent to the nodes
        $ns attach-agent $n0 $tcp_src1
        $ns attach-agent $n1 $tcp_snk1
        $ns attach-agent $n1 $tcp_src2
        $ns attach-agent $n2 $tcp_snk2
        $ns attach-agent $n2 $tcp_src3
        $ns attach-agent $n0 $tcp_snk3
```

#connect source and sink

```
$ns connect $tcp_src1 $tcp_snk1
$ns connect $tcp_src2 $tcp_snk2
$ns connect $tcp_src3 $tcp_snk3
```

Create ftp sources at the each node set

```
ftp1 [$tcp_src1 attach-source FTP]
set ftp2 [$tcp_src2 attach-source FTP]
set ftp3 [$tcp_src3 attach-source FTP]
```

Start up the first ftp at the time 0 and the second ftp staggered 1 second later

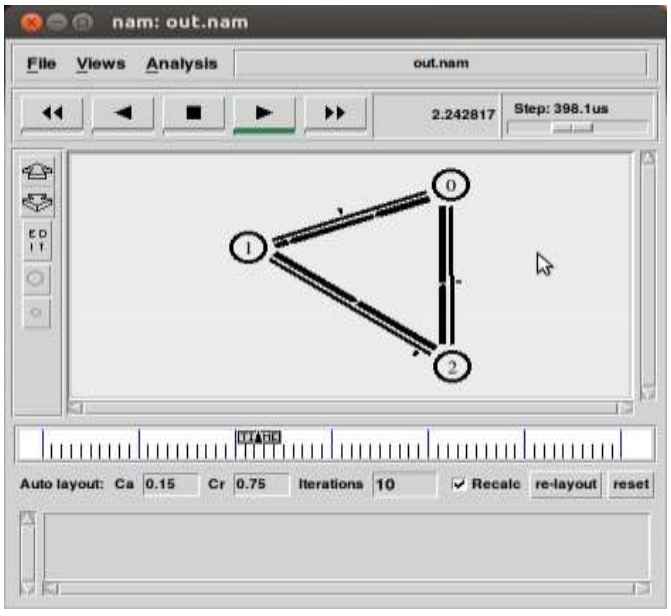
```
$ns at 0.0 "$ftp1 start"
$ns at 4.0 "$ftp1 stop"
$ns at 1.0 "$ftp2 start"
$ns at 6.0 "$ftp2 stop"
$ns at 2.0 "$ftp3 start"
$ns at 5.0 "$ftp3 stop"
```

run the simulation for 10 simulated seconds

```
$ns at 10.0 "exit 0"
$ns run
```

----------OUTPUT---------- ---------- $

ns p1.tcl
$ nam out.nam
$ grep –c tcp out.tr **#number of packets sent**
13782
$ grep ^d out.tr | wc –l **#number of packets dropped**
56



Program No. 2

Simulate the transmission of ping messages over a network topology consisting of 6 nodes & find the number of packets dropped due to congestion.

P2.tcl

```
set ns [new Simulator]

        set n1 [$ns node]  set
        n2 [$ns node]  set n3
        [$ns node]  set n4
        [$ns node]  set n5
        [$ns node]  set n6
        [$ns node]

        set f [open out.tr w]
        $ns trace-all $f

        set nf [open out.nam w]
        $ns namtrace-all $nf

        $ns duplex-link $n1 $n3 10Mb 10ms DropTail
        $ns duplex-link $n1 $n2 10Mb 10ms DropTail
        $ns duplex-link $n3 $n4 10Mb 10ms DropTail
        $ns duplex-link $n4 $n5 10Mb 10ms DropTail
        $ns duplex-link $n4 $n6 10Mb 10ms DropTail

# connect the ping agents  set
        ping0 [new Agent/Ping]
        $ns attach-agent $n1 $ping0
        set ping1 [new Agent/Ping]
        $ns attach-agent $n6 $ping1
        set ping2 [new Agent/Ping]
        $ns attach-agent $n2 $ping2
        set ping3 [new Agent/Ping]
        $ns attach-agent $n5 $ping3

        $ns connect $ping0 $ping1
        $ns connect $ping2 $ping3

#The below function is executed when the ping agent receives #a reply from the destination
        Agent/Ping instproc recv {from rtt} { $self instvar node_
        puts " The node [$node_ id] received an reply from $from with round trip time of $rtt"  }

# to start sending the ping packets
        $ns at 0.1 "$ping0 send"
        $ns at 0.2 "$ping0 send"
```

```
$ns at 0.3 "$ping0 send"
$ns at 0.4 "$ping0 send"
$ns at 0.5 "$ping0 send"
$ns at 0.6 "$ping0 send"
$ns at 0.7 "$ping0 send"
$ns at 0.8 "$ping0 send"
$ns at 0.9 "$ping0 send"
$ns at 1.0 "$ping0 send"

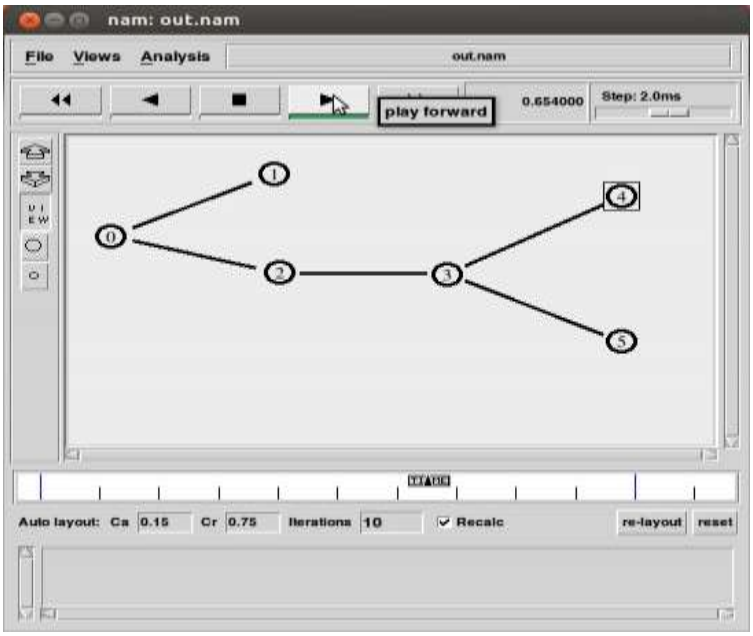
$ns at 0.1 "$ping2 send"
$ns at 0.2 "$ping2 send"
$ns at 0.3 "$ping2 send"
$ns at 0.4 "$ping2 send"
$ns at 0.5 "$ping2 send"
$ns at 0.6 "$ping2 send"
$ns at 0.7 "$ping2 send"
$ns at 0.8 "$ping2 send"
$ns at 0.9 "$ping2 send"
$ns at 1.0 "$ping2 send"

$ns at 6.0 "exit 0"
$ns run
```

----------OUTPUT---------- $ns p3.tcl

The node 0 received an reply from 5 with round trip time of 60.3
The node 1 received an reply from 4 with round trip time of 80.4

$nam out.nam



Program No.03

Simulate an Ethernet LAN using n nodes & set multiple traffic nodes & plot congestion window for different source/destination.

P3.tcl

```
set ns [new Simulator]

        set n0 [$ns node]
set n1 [$ns node]       set n2
[$ns node]      set n3 [$ns
node]           set n4 [$ns
node]           set n5 [$ns
node]           set n6 [$ns
node]

        set f [open out.tr w]
$ns trace-all $f        set nf
[open out.nam w]
        $ns namtrace-all $nf

#create graph files
        set winFile0 [open winFile0 w]
set winFile1 [open winFile1 w]

        $ns make-lan "$n0 $n1 $n2 $n3" 10Mb 10ms LL Queue/DropTail Mac/802_3

        $ns duplex-link $n3 $n4 10Mb 10ms DropTail
        $ns duplex-link $n4 $n6 10Mb 10ms DropTail
        $ns duplex-link $n4 $n5 10Mb 10ms DropTail

        $ns duplex-link-op $n3 $n4 orient right-down
        $ns duplex-link-op $n4 $n6 orient right-down
        $ns duplex-link-op $n4 $n5 orient left-down

        $ns queue-limit $n4 $n6 5

        set tcp_src1 [new Agent/TCP]
set tcp_snk1 [new Agent/TCPSink]   set
tcp_src2 [new Agent/TCP]
        set tcp_snk2 [new Agent/TCPSink]

        $ns attach-agent $n0 $tcp_src1
        $ns attach-agent $n5 $tcp_snk1
        $ns attach-agent $n1 $tcp_src2
        $ns attach-agent $n6 $tcp_snk2
```

```
        $ns connect $tcp_src1 $tcp_snk1
        $ns connect $tcp_src2 $tcp_snk2

        set ftp1 [$tcp_src1 attach-source FTP]
set ftp2 [$tcp_src2 attach-source FTP]

#plot window proc
        proc PlotWindow {tcpSource file}  {
                global ns                set time
0.1             set now [$ns now]                set
cwnd [$tcpSource set cwnd_ ]
puts $file "$now $cwnd"
                $ns at [expr $now+$time] "PlotWindow $tcpSource $file"
        }

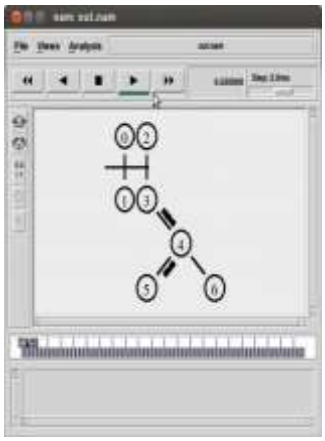
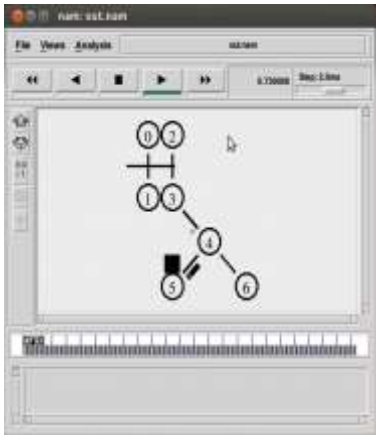
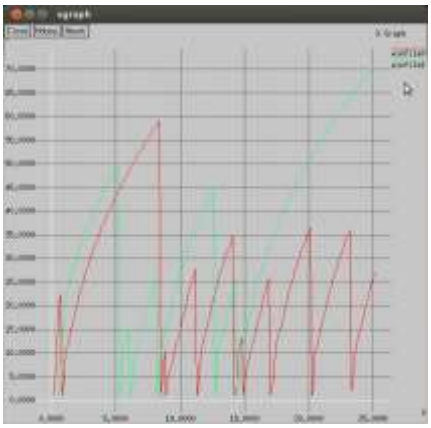
        set err [new ErrorModel]
$ns lossmodel $err $n3 $n4

#inserting the error rate
        $err set rate_ 0.001

        $ns at 0.1 "$ftp1 start"
        $ns at 0.1 "PlotWindow $tcp_src1 $winFile0"
        $ns at 0.5 "$ftp2 start"
        $ns at 0.1 "PlotWindow $tcp_src2 $winFile1"
        $ns at 25.0 "$ftp1 stop"
        $ns at 25.1 "$ftp2 stop"
        $ns at 25.2 "exit 0"
        $ns run
```

----------OUTPUT----------

```
$ ns p5.tcl
$ nam out.nam
$ xgraph winfile0 winfile1          #to plot the graph
```

Program No. 04
Write a program for error detecting code using CRC-CCITT(16-bits).

Java is a general-purpose computer programming language that is simple, concurrent, class-based, object-oriented language. The compiled Java code can run on all platforms that support Java without the need for recompilation hence Java is called as "write once, run anywhere" (WORA).The Java compiled intermediate output called “byte-code” that can run on any Java virtual machine (JVM) regardless of computer architecture. The language derives much of its syntax from C and C++, but it has fewer low-level facilities than either of them. In Linux operating system Java libraries are preinstalled. It’s very easy and convenient to compile and run Java programs in Linux environment. To compile and run Java Program is a two-step process: 1. Compile Java Program from Command Prompt [root@host ~]# javac Filename.java The Java compiler (Javac) compiles java program and generates a byte-code with the same file name and .class extension. 2. Run Java program from Command Prompt [root@host ~]# java Filename The java interpreter (Java) runs the byte-code and gives the respective output. It is important to note that in above command we have omitted the .class suffix of the bytecode (Filename.class).

Whenever digital data is stored or interfaced, data corruption might occur. Since the beginning of computer science, developers have been thinking of ways to deal with this type of problem. For serial data they came up with the solution to attach a parity bit to each sent byte. This simple detection mechanism works if an odd number of bits in a byte changes, but an even number of false bits in one byte will not be detected by the parity check. To overcome this problem developers have searched for mathematical sound mechanisms to detect multiple false bits. The CRC calculation or cyclic redundancy check was the result of this. Nowadays CRC calculations are used in all types of communications. All packets sent over a network connection are checked with a CRC. Also each data block on your hard disk has a CRC value attached to it. Modern computer world cannot do without these CRC calculations. So let's see why they are so widely used. The answer is simple; they are powerful, detect many types of errors and are extremely fast to calculate especially when dedicated hardware chips are used.

The idea behind CRC calculation is to look at the data as one large binary number. This number is divided by a certain value and the remainder of the calculation is called the CRC. Dividing in the CRC calculation at first looks to cost a lot of computing power, but it can be performed very quickly if we use a method similar to the one learned at school. We will as an example calculate the remainder for the character 'm'— which is 1101101 in binary notation— by dividing it by 19 or 10011. Please note that 19 is an odd number. This is necessary as we will see further on. Please refer to your schoolbooks as the binary calculation method here is not very different from the decimal method you learned when you were young. It might only look a little bit strange. Also notations differ between countries, but the method is similar.

```
                    1 0 1 = 5
                 -------------
1 0 0 1 1 / 1 1 0 1 1 0 1
            1 0 0 1 1 | |
            --------- | |
              1 0 0 0 0 |
              0 0 0 0 0 |
              --------- |
              1 0 0 0 0 1
                1 0 0 1 1
                ---------
                  1 1 1 0 = 14 = remainder
```

With decimal calculations you can quickly check that 109 divided by 19 gives a quotient of 5 with 14 as the remainder. But what we also see in the scheme is that every bit extra to check only costs one binary comparison and in 50% of the cases one binary subtraction. You can easilyincrease the number of bits of the test data string—for example to 56 bits if we use our example value "Lammert"—and the result can be calculated with 56 binary comparisons and an average of 28 binary subtractions. This can be implemented in hardware directly with only very few transistors involved. Also software algorithms can be very efficient. All of the CRC formulas you will encounter are simply checksum algorithms based on modulo-2 binary division where we ignore carry bits and in effect the subtraction will be equal to an exclusive or operation. Though some differences exist in the specifics across different CRC formulas, the basic mathematical process is always the same: • The message bits are appended with c zero bits; this augmented message is the dividend • A predetermined c+1-bit binary sequence, called the generator polynomial, is the divisor • The checksum is the c-bit remainder that results from the division operation.

Table 1 lists some of the most commonly used generator polynomials for 16- and 32-bit CRCs. Remember that the width of the divisor is always one bit wider than the remainder. So, for example, you'd use a 17-bit generator polynomial whenever a 16-bit checksum is required.

	CRC-CCITT	CRC-16	CRC-32
Checksum Width	16 bits	16 bits	32 bits
Generator Polynomial	10001000000100001	11000000000000101	100000100110000010001110110110111

International Standard CRC Polynomials

#code

```
import java.io.*;
class Crc {
public static void main(String args[]) throws IOException
{
BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
int[ ] data; int[ ]div; int[ ]divisor; int[ ]rem; int[ ] crc;
int data_bits, divisor_bits, tot_length;
System.out.println("Enter number of data bits : ");
data_bits=Integer.parseInt(br.readLine()); data=new
int[data_bits];
System.out.println("Enter data bits : "); for(int
i=0; i<data_bits; i++)
data[i]=Integer.parseInt(br.readLine());
System.out.println("Enter number of bits in divisor : ");
divisor_bits=Integer.parseInt(br.readLine()); divisor=new
int[divisor_bits];
System.out.println("Enter Divisor bits : ");
for(int i=0; i<divisor_bits; i++)
divisor[i]=Integer.parseInt(br.readLine()); /*
System.out.print("Data bits are : "); for(int
i=0; i< data_bits; i++)
System.out.print(data[i]);
System.out.println();
System.out.print("divisor bits are : "); for(int
i=0; i< divisor_bits; i++)
System.out.print(divisor[i]);
System.out.println();
```

```
*/ tot_length=data_bits+divisor_bits-1;
div=new int[tot_length]; rem=new
int[tot_length]; crc=new
int[tot_length];
/*------------------ CRC GENERATION----------------------*/
for(int i=0;i<data.length;i++) div[i]=data[i];
System.out.print("Dividend (after appending 0's) are : "); for(int
i=0; i< div.length; i++)
System.out.print(div[i]);
System.out.println(); for(int
j=0; j<div.length; j++){ rem[j]
= div[j];
}
rem=divide(div, divisor, rem);
for(int i=0;i<div.length;i++) //append dividend and ramainder
{
crc[i]=(div[i]^rem[i]);
}
System.out.println();
System.out.println("CRC code : "); for(int
i=0;i<crc.length;i++)
System.out.print(crc[i]);
/*-------------------ERROR DETECTION----------------------*/
System.out.println();
System.out.println("Enter CRC code of "+tot_length+" bits : ");
for(int i=0; i<crc.length; i++)
crc[i]=Integer.parseInt(br.readLine()); /*
System.out.print("crc bits are : ");
for(int i=0; i< crc.length; i++)
System.out.print(crc[i]);
System.out.println();
*/
for(int j=0; j<crc.length; j++)
{
rem[j] = crc[j];
} rem=divide(crc, divisor,
rem); for(int i=0; i<
rem.length; i++)
{ if(rem[i]!=0)
{
System.out.println("Error");
break; }
if(i==rem.length-1) System.out.println("No
Error");
}
System.out.println("THANK YOU.... :)");
}
```

```
static int[] divide(int div[],int divisor[], int rem[])
{ int
cur=0;
while(true)
{
for(int i=0;i<divisor.length;i++)
rem[cur+i]=(rem[cur+i]^divisor[i]);
while(rem[cur]==0 && cur!=rem.length-1) cur++;
if((rem.length-cur)<divisor.length)
break; } return rem;
}
}
```

Output



Program No.05

Develop a program to implement a sliding window protocol in the data link layer.

```
#include<stdio.h> int
main ()
{
        int w,i,f,frames[50];
printf("Enter window size: ");
scanf("%d",&w);
        printf("\nEnter number of frames to transmit: ");
        scanf("%d",&f);
        printf("\nEnter %d frames: ",f);
        for(i=1;i<=f;i++)
scanf("%d",&frames[i]);
        printf("\nWith sliding window protocol the frames will be sent in the following manner (assuming
no corruption of frames)\n\n");
        printf("After sending %d frames at each stage sender waits for acknowledgement sent by
the receiver\n\n",w);
        for(i=1;i<=f;i++)
        {
                if(i%w==0)
                {
                        printf("%d\n",frames[i]);
                        printf("Acknowledgement of above frames sent is received by sender\n\n");
                }
                else
                        printf("%d ",frames[i]);
        }
         if(f%w!=0)
         printf("\nAcknowledgement of above frames sent is received by sender\n");
return 0;
}
```

OUTPUT:

```
Enter window size: 3
Enter number of frames to transmit: 5
Enter 5 frames: 12 5 89 4 6
With sliding window protocol the frames will be sent in the following manner (assuming no corruption of
frames)
12 5 89
Acknowledgement of above frames sent is received by sender
4 6
Acknowledgement of above frames sent is received by sender
```

Program No.06

Develop program to find shortest path between vertices using Bellman ford & Path vector routing algorithm

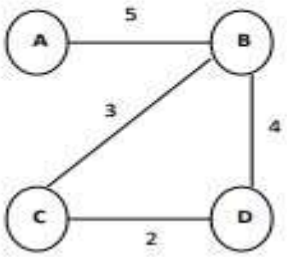
Distance Vector Algorithm is a decentralized routing algorithm that requires that each router simply inform its neighbors of its routing table. For each network path, the receiving routers pick the neighbor advertising the lowest cost, then add this entry into its routing table for re-advertisement. To find the shortest path, Distance Vector Algorithm is based on one of two basic algorithms: the Bellman-Ford and the Dijkstra algorithms. Routers that use this algorithm have to maintain the distance tables (which is a onedimension array -- "a vector"), which tell the distances and shortest path to sending packets to each node in the network. The information in the distance table is always up date by exchanging information with the neighboring nodes. The number of data in the table equals to that of all nodes in networks (excluded itself). The columns of table represent the directly attached neighbors whereas the rows represent all destinations in the network. Each data contains the path for sending packets to each destination in the network and distance/or time to transmit on that path (we call this as "cost"). The measurements in this algorithm are the number of hops, latency, the number of outgoing packets, etc. The Bellman–Ford algorithm is an algorithm that computes shortest paths from a single source vertex to all of the other vertices in a weighted digraph. It is slower than Dijkstra's algorithm for the same problem, but more versatile, as it is capable of handling graphs in which some of the edge weights are negative numbers. Negative edge weights are found in various applications of graphs, hence the usefulness of this algorithm. If a graph contains a "negative cycle" (i.e. a cycle whose edges sum to a negative value) that is reachable from the source, then there is no cheapest path: any path that has a point on the negative cycle can be made cheaper by one more walk around the negative cycle. In such a case, the Bellman–Ford algorithm can detect negative cycles and report their existence. // distance

```
import java.util.Scanner; public
class BellmanFord
{
private int D[]; private
int num_ver;
public static final int MAX_VALUE = 999; public
BellmanFord(int num_ver)
{
this.num_ver = num_ver; D
= new int[num_ver + 1];
}
public void BellmanFordEvaluation(int source, int A[][])
{
for (int node = 1; node <= num_ver; node++)
{
D[node] = MAX_VALUE;
}
D[source] = 0;
for (int node = 1; node <= num_ver - 1; node++)
{
for (int sn = 1; sn <= num_ver; sn++)
{
for (int dn = 1; dn <= num_ver; dn++)
```

```
{
if (A[sn][dn] != MAX_VALUE)
{
if (D[dn] > D[sn]+ A[sn][dn])
D[dn] = D[sn] + A[sn][dn];
}
}
} }
for (int sn = 1; sn <= num_ver; sn++)
{
for (int dn = 1; dn <= num_ver; dn++)
{
if (A[sn][dn] != MAX_VALUE)
{
if (D[dn] > D[sn]+ A[sn][dn])
System.out.println("The Graph contains negative egde cycle");
}
} }
for (int vertex = 1; vertex <= num_ver; vertex++)
{
System.out.println("distance of source " + source + " to "+ vertex + " is
" + D[vertex]);
} }
public static void main(String[ ] args)
{ int num_ver =
0; int source;
Scanner scanner = new Scanner(System.in);
System.out.println("Enter the number of vertices");
num_ver = scanner.nextInt();
int A[][] = new int[num_ver + 1][num_ver + 1];
System.out.println("Enter the adjacency matrix");
for (int sn = 1; sn <= num_ver; sn++)
{
for (int dn = 1; dn <= num_ver; dn++)
{
A[sn][dn] = scanner.nextInt();
if (sn == dn) {
A[sn][dn] = 0;
continue; }
if (A[sn][dn] == 0)
{
A[sn][dn] = MAX_VALUE;
}
}
}
System.out.println("Enter the source vertex");
source = scanner.nextInt();
BellmanFord b = new BellmanFord (num_ver);
b.BellmanFordEvaluation(source, A); scanner.close();
}
}

//output
```

Input graph:




```
root@localhost:~
File  Edit  View  Terminal  Help
[root@localhost ~]# javac BellmanFord.java
[root@localhost ~]# java BellmanFord
Enter the number of vertices
4
Enter the adjacency matrix
0 5 0 0
5 0 3 4
0 3 0 2
0 4 2 0
Enter the source vertex
2
distance of source  2 to 1 is 5
distance of source  2 to 2 is 0
distance of source  2 to 3 is 3
distance of source  2 to 4 is 4
[root@localhost ~]#
```

Program No 7

Using TCP or IP sockets write a client/server program to make client send the name of a file and server to send back the contents of the requested file if present.

Socket is an interface which enables the client and the server to communicate and pass on information from one another. Sockets provide the communication mechanism between two computers using TCP. A client program creates a socket on its end of the communication and attempts to connect that socket to a server. When the connection is made, the server creates a socket object on its end of the communication. The client and the server can now communicate by writing to and reading from the socket.

```
//TCP Client import
java.io.BufferedReader; import
java.io.DataInputStream;  import
java.io.DataOutputStream;  import
java.io.EOFException; import
java.io.File; import
java.io.FileOutputStream; import
java.io.InputStreamReader; import
java.net.Socket; import
java.util.Scanner;  public class
Client
{
        public static void main(String args[])throws Exception
          {
            String address = "";
            Scanner sc=new Scanner(System.in);
System.out.println("Enter Server Address: ");
address=sc.nextLine();              //create the socket on
port 7000
            Socket s=new Socket(address,7000);
            DataInputStream din=new DataInputStream(s.getInputStream());
            DataOutputStream dout=new DataOutputStream(s.getOutputStream());
            BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
            System.out.println("Send Get to start...");
            String str="",filename="";
            try
            {
              while(!str.equals("start"))
str=br.readLine();
dout.writeUTF(str);
              dout.flush();
filename=din.readUTF();
              System.out.println("Receving file: "+filename);
filename="client"+filename;
```

```
                System.out.println("Saving as file: "+filename);
        long sz=Long.parseLong(din.readUTF());
                System.out.println ("File Size: "+(sz/(1024*1024))+" MB");
                byte b[]=new byte [1024];
                System.out.println("Receving file..");
                FileOutputStream fos=new FileOutputStream(new
File(filename),true);          long bytesRead;          do                  {
                  bytesRead = din.read(b, 0, b.length);
fos.write(b,0,b.length);
                }
                while(!(bytesRead<1024));
System.out.println("Comleted");
fos.close();              dout.close();
s.close();
              }
              catch(EOFException e)
                {
                 //do nothing
                }
            }
          }
```

TCP Server

```
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.File; import
java.io.FileInputStream; import
java.net.ServerSocket; import
java.net.Socket;
import java.util.Scanner;

public class Server {

          public static void main(String[] args) throws Exception
          {
            String filename;
            System.out.println("enter file
name");            Scanner sc=new
Scanner(System.in);
filename=sc.nextLine();          sc.close();
while(true)
            {
                  ServerSocket ss=new ServerSocket(5000);
                  System.out.println("waiting for request");
                  Socket s=ss.accept();
                  System.out.println("connected with"+s.getInetAddress().toString());
```

```
                DataInputStream din=new DataInputStream(s.getInputStream());
DataOutputStream dout =new DataOutputStream(s.getOutputStream());
                try
                {
                        String str="";
                        str=din.readUTF();
                        System.out.println("sendget------ok");
                        if(!str.equals("stop"))
                        {
                                System.out.println("sending
file:"+filename);                                       dout.writeUTF(filename);
                dout.flush();
                                File f=new File(filename);
                                FileInputStream fin=new FileInputStream(f);
                                long sz=(int)f.length();
                                byte[] b=new byte[1024];
                int read;
                                dout.writeUTF(Long.toString(sz));
        dout.flush();
                                System.out.println("size:"+sz);
                                System.out.println("buf size:"+ss.getReceiveBufferSize());
                                while((read=fin.read(b))!=-1)
                                {
                                        dout.write(b,0,read);
                                        dout.flush();
                                }
                                fin.close();
                                System.out.println("-----ok");
                dout.flush();
                        }
                        dout.writeUTF("stop");
                        System.out.println("send complete");
        dout.flush();
                }
                catch(Exception e)
                {
                        e.printStackTrace();
                        System.out.println("an error occured");
                }
                sc.close();
                ss.close();
        }

       }

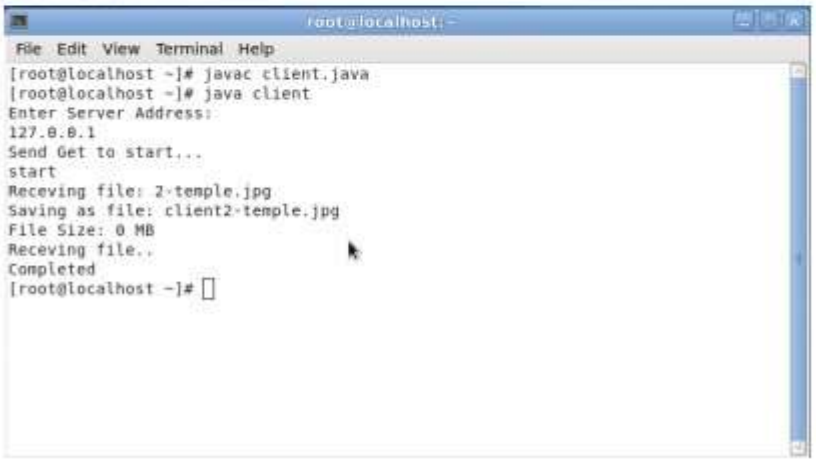
}
```

Output:

At server side:



At client side:



Program No.8

Write a program on datagram socket for client/server to display the messages on client side, typed at the server side.

A datagram socket is the one for sending or receiving point for a packet delivery service. Each packet sent or received on a datagram socket is individually addressed and routed. Multiple packets sent from one machine to another may be routed differently, and may arrive in any order.

```
//10.udpc client

//UDP Client

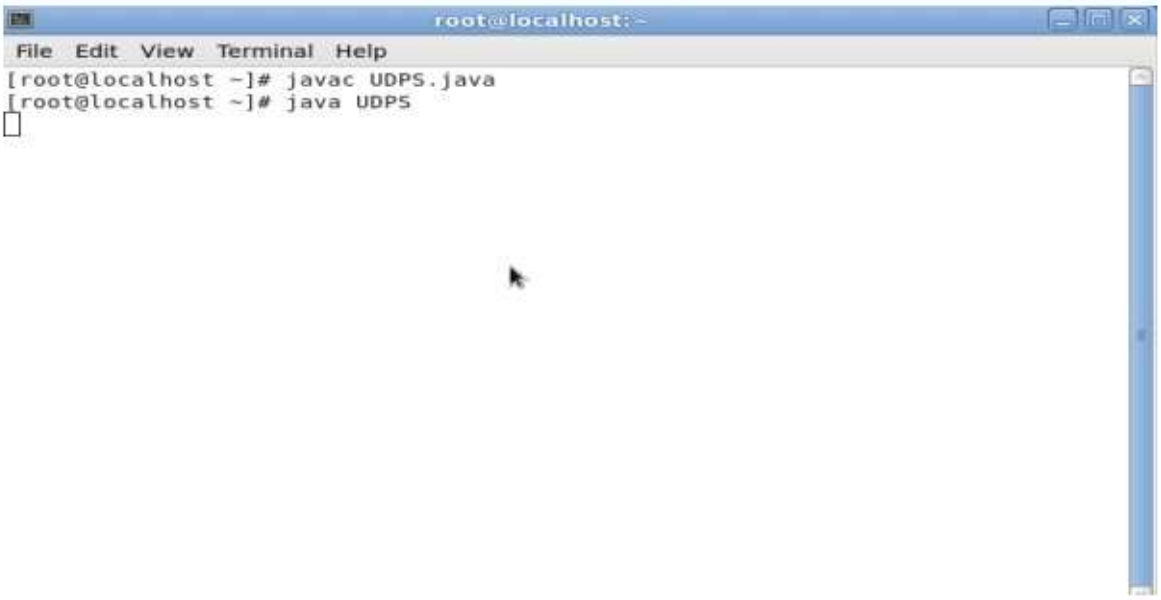
import java.io.*;
import java.net.*;
import java.net.InetAddress;
class UDPClient
{
public static void main(String args[]) throws Exception
{
BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
DatagramSocket clientSocket = new DatagramSocket();
InetAddress IPAddress = InetAddress.getByName("localhost");
byte[] sendData = new byte[1024];
byte[] receiveData = new byte[1024];
String sentence = "Hello Server";
sendData = sentence.getBytes();
DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, 9876);
clientSocket.send(sendPacket);
DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
clientSocket.receive(receivePacket);
String modifiedSentence = new String(receivePacket.getData());
System.out.println("FROM SERVER:" + modifiedSentence);
clientSocket.close();
}
}

//UDP Server

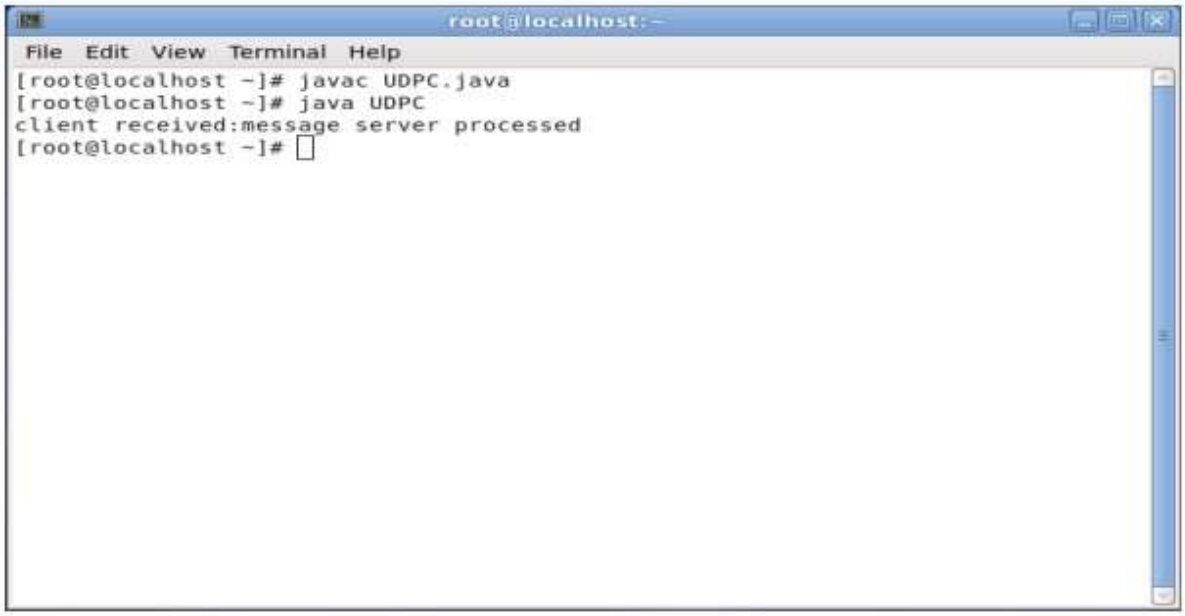
import java.io.*;
import java.net.*;
import java.net.InetAddress;
class UDPServer
{
public static void main(String args[]) throws Exception
{
DatagramSocket serverSocket = new DatagramSocket(9876);
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
byte[] receiveData = new byte[1024];
```

```
byte[] sendData = new byte[1024];
DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
serverSocket.receive(receivePacket);
String sentence = new String( receivePacket.getData());
System.out.println("RECEIVED: " + sentence);
InetAddress IPAddress = receivePacket.getAddress(); int port = receivePacket.getPort();
System.out.println("Enter the Message");
String data = br.readLine(); sendData = data.getBytes();
DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, port);
serverSocket.send(sendPacket); serverSocket.close();
}
}
```

OUTPUT



At Client side:



Program No.9

Write a program for simple RSA algorithm to encrypt and decrypt the data.

RSA is an example of public key cryptography. It was developed by Rivest, Shamir and Adelman. The RSA algorithm can be used for both public key encryption and digital signatures. Its security is based on the difficulty of factoring large integers. The RSA algorithm's efficiency requires a fast method for performing the modular exponentiation operation. A less efficient, conventional method includes raising a number (the input) to a power (the secret or public key of the algorithm, denoted e and d, respectively) and taking the remainder of the division with N. A straight-forward implementation performs these two steps of the operation sequentially: first, raise it to the power and second, apply modulo. The RSA algorithm comprises of three steps, which are depicted below: Key Generation Algorithm 1. Generate two large random primes, p and q, of approximately equal size such that their product n = p*q 2. Compute n = p*q and Euler's totient function (φ) phi(n) = (p-1)(q-1). 3. Choose an integer e, 1 < e < phi, such that gcd(e, phi) = 1. 4. Compute the secret exponent d, 1 < d < phi, such that e*d ≡ 1 (mod phi). 5. The public key is (e, n) and the private key is (d, n). The values of p, q, and phi should also be kept secret. Encryption Sender A does the following:-

1. Using the public key (e,n)
2. Represents the plaintext message as a positive integer M
3. Computes the cipher text C = M^e mod n.
4. Sends the cipher text C to B (Receiver).

Decryption Recipient B does the following:-

1. Uses his private key (d, n) to compute M = C^d mod n.
2. Extracts the plaintext from the integer representative m.

```
//code
import java.io.DataInputStream;
import java.io.IOException;
import java.math.BigInteger;
import java.util.Random; public
class RSA
{
private BigInteger p;
private BigInteger q;
private BigInteger N;
private BigInteger phi;
private BigInteger e; private
BigInteger d; private int
bitlength = 1024; private
Random r;
public RSA()
{
        r = new Random();
```

```
        p = BigInteger.probablePrime(bitlength, r); q
        = BigInteger.probablePrime(bitlength, r); N
        = p.multiply(q);
        phi = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE)); e
        = BigInteger.probablePrime(bitlength / 2, r);
        while (phi.gcd(e).compareTo(BigInteger.ONE) > 0 && e.compareTo(phi) < 0)
        {
                e.add(BigInteger.ONE);
        }
        d = e.modInverse(phi);
}
public RSA(BigInteger e, BigInteger d, BigInteger N)
{ this.e = e; this.d
        = d;
        this.N = N;
}
@SuppressWarnings("deprecation")
public static void main(String[] args) throws IOException
{
        RSA rsa = new RSA();
        DataInputStream in = new DataInputStream(System.in);
        String teststring;
        System.out.println("Enter the plain text:");
        teststring = in.readLine();
        System.out.println("Encrypting String: " + teststring);
        System.out.println("String in Bytes: "
        + bytesToString(teststring.getBytes()));
        // encrypt
        byte[] encrypted = rsa.encrypt(teststring.getBytes());
        // decrypt
        byte[] decrypted = rsa.decrypt(encrypted);
        System.out.println("Decrypting Bytes: " + bytesToString(decrypted));
        System.out.println("Decrypted String: " + new String(decrypted));
}
private static String bytesToString(byte[] encrypted)
{
        String test = "";
        for (byte b : encrypted)
        {
                test += Byte.toString(b);
        }

return test;
```

```
}
// Encrypt message
public byte[] encrypt(byte[] message)
{
        return (new BigInteger(message)).modPow(e, N).toByteArray();
}
// Decrypt message
public byte[] decrypt(byte[] message)
{
        return (new BigInteger(message)).modPow(d, N).toByteArray();
}
}
```

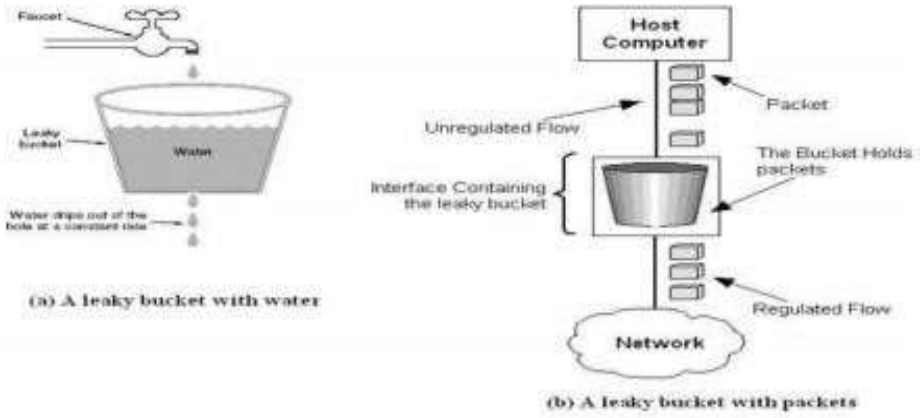
Output:

```
Enter the plain text:
hi bangalore
Encrypting String: hi bangalore
String in Bytes: 104105329897110103971081111141 01
Decrypting Bytes: 104105329897110103971081111141 01
Decrypted String: hi bangalore
```

Program No 10

Program for congestion control using Leaky Bucket algorithm.

The main concept of the leaky bucket algorithm is that the output data flow remains constant despite the variant input traffic, such as the water flow in a bucket with a small hole at the bottom. In case the bucket contains water (or packets) then the output flow follows a constant rate, while if the bucket is full any additional load will be lost because of spillover. In a similar way if the bucket is empty the output will be zero. From network perspective, leaky bucket consists of a finite queue (bucket) where all the incoming packets are stored in case there is space in the queue, otherwise the packets are discarded. In order to regulate the output flow, leaky bucket transmits one packet from the queue in a fixed time (e.g. at every clock tick). In the following figure we can notice the main rationale of leaky bucket algorithm, for both the two approaches (e.g. leaky bucket with water (a) and with packets (b)).



(a) A leaky bucket with water

(b) A leaky bucket with packets

```
import java.lang.*; import java.util.Random; import java.io.*; import
java.util.Scanner; class leaky_bucket
{
public static void main(String args[])
{ int drop=0,mini,nsec,p_remain=0;  int
        o_rate,b_size,i,packet[]; packet =
        new int[100];
        Scanner in = new Scanner(System.in);
        System.out.println("Enter bucket size:");
        b_size = in.nextInt();
        System.out.println("Enter the output rate:");
        o_rate = in.nextInt();
        System.out.println("Enter the number of seconds you want to simulate:");  nsec
        = in.nextInt();
        Random rand = new Random();
        for(i=0;i<nsec;i++)
        packet[i]=((rand.nextInt(9)+1)*10);
        System.out.println("Seconds|packets received|packets sent|packets left|packets dropped");
        System.out.println(" ");
        for(i=0;i<nsec;i++)
        {
                p_remain+=packet[i];
                if(p_remain>b_size)
                {
                        drop=p_remain-b_size;
                        p_remain=b_size;
                        System.out.print(i+1+" ");
                        System.out.print(packet[i]+" ");
                        mini=Math.min(p_remain,o_rate);
                        System.out.print(mini+" ");
                        p_remain=p_remain-mini;
                        System.out.print(p_remain+" ");
                        System.out.print(drop+" ");
                        System.out.println();
                        drop=0;
                }
        }
        while(p_remain!=0)
        {
                if(p_remain>b_size)
                {
                        drop=p_remain-b_size; p_remain=b_size;
                }
```

```
                mini=Math.min(p_remain,o_rate);
                System.out.print(" "+p_remain+" "+mini); p_remain=p_remain-mini;
                System.out.println(p_remain+" "+drop); drop=0;
        }
}
}
```

Output:

```
C:\Users\abhijith\Desktop>javac leaky_bucket.java

C:\Users\abhijith\Desktop>java leaky_bucket
Enter bucket size:
10
Enter the output rate:
4
Enter the number of seconds you want to simulate:
10
Seconds|packets received|packets sent|packets left|packets dropped
-------------------------------------------------------------------------------
1               90              4               6               80
2               20              4               6               16
3               50              4               6               46
4               90              4               6               86
5               70              4               6               66
6               40              4               6               36
7               70              4               6               66
8               90              4               6               86
9               10              4               6               6
10              70              4               6               66
                6               42              0
                2               20              0
```